

AI Engineer - 2nd interview task

Overview

Your goal is to implement an application in Node.js/TypeScript according to the following description. You can implement everything before the meeting, or have it partially done with the plan for the rest.

During the meeting, you will be asked to share your screen and keep your camera on. You will collaborate with other engineers - discussing the task, implementing the logic, and answering questions.

You can use any tool you will normally use during the task implementation (e.g. documentation, co-pilot etc.).

The final goal is to have a fully operational application.

Task description

Implement in Node.js/TypeScript an agent system that auto-generates a short test on a customizable topic. The agent system should then run the quiz, collect the user's answers, and calculate the final score. The app should be a small production-ready app, not a PoC.

The quiz knowledge should be fetched from a Markdown file, provided to the application via configurable URL (e.g. <https://github.com/pipeocat-ai/pipeocat/blob/main/README.md>).

The test should consist of 5 to 8 questions, each with a closed list of 4 possible answers.

The agent should score each user's answer with following rules:

- 4 points - correct answer
- 0 points - wrong answer

- between 0 and 4 - number of correctly selected answers in case of multiple answers question

The final score is calculated as a weighted average of all individual scores. The weight for each answer is a value in a geometric sequence starting from 1.0, with each subsequent weight increased by 10%.

Requirements:

- The agent should store questions, answers, and the final score in a database. *Be prepared to explain the data flow & data modelling process to us.*
- Run the quiz on at least 2 different README.md
- You may use any library or framework (e.g. Langchain, Mastra, etc.) you want. You can use any free-tier LLM (e.g. Groq, OpenRouter, ollama).
- You should include a web UI for running the quiz. You can use any modern FE framework (e.g. React).
- You should expose the agent via the REST API (or similar)

Nice to have:

- LLM observability (e.g. Langfuse)
- Different strategies for question generation